

PUCE SEDE MANABÍ • INGENIERÍA DE SOFTWARE

Desarrollo de Sistemas de Información • 2026-1

PortoSinFiltro

El muro público de Portoviejo

Adolfo Castro • Axel Hernández • Elkin Saltos

Marcelo Morales • Johan Medranda





Una vía sin reparar deja de ser una molestia y pasa a ser un riesgo.

02 • CONTEXTO

Denunciar no sirve si nadie lo ve

El ciudadano de Portoviejo reporta un bache o un poste apagado y el reclamo desaparece: no hay número de caso, no hay quién más lo respalde, no hay forma de saber si avanzó.

1

Sin visibilidad

El reclamo muere en una llamada o un mensaje privado. Nadie más se entera de que el problema existe.

2

Sin presión

Un vecino solo es fácil de ignorar. Treinta vecinos respaldando el mismo caso, no.

3

Sin cierre

Nadie confirma si de verdad se arregló. El «resuelto» oficial no siempre coincide con la calle.

Un muro público donde manda la comunidad

El estado de una denuncia no lo cambia un funcionario: lo calcula la evidencia ciudadana.

VISIBILIDAD

Muro público

Cualquiera lee las denuncias sin iniciar sesión, con foto, mapa, zona y gravedad.

RESPALDO

Apoyo ciudadano

Apoyos, aportes y reportes de contenido falso: un voto por persona, verificado.

CIERRE VERIFICADO

3 vecinos

Una denuncia solo pasa a RESUELTA cuando tres ciudadanos distintos lo confirman.

La regla que define el producto

Ninguna autoridad participa: no hay municipio, ni cuadrillas, ni resolución oficial. La plataforma es ciudadana e independiente, y por eso el cierre de un caso tiene que venir de los vecinos.

Ver → Denunciar → Apoyar → Resolver

01 • VISITANTE

Lee el muro

Sin cuenta. Filtra por estado, zona y categoría; ordena por reciente, apoyos o gravedad.

02 • CIUDADANO

Publica

Categoría, zona, gravedad 1-5, punto exacto en el mapa y foto. Puede firmar o ir anónimo.

03 • COMUNIDAD

Respalda

Apoya, aporta evidencia, vota si el caso progresa o lo reporta como falso.

04 • COMUNIDAD

Cierra

Tres confirmaciones independientes y el estado cambia solo, en vivo, sin recargar.

05 • ADMIN

Modera

Solo oculta o restaura lo abusivo. Nunca decide si un problema está resuelto.

Lo que verán en la demostración

Crear una denuncia anónima • apoyarla desde otra cuenta • ver el muro actualizarse solo • moderarla desde el panel de administración. Todo en producción, no en un entorno local.

Scrum adaptado a un semestre corto

Por qué Scrum y no Kanban

El alcance estaba fijo por la rúbrica y la fecha de examen era inamovible: necesitábamos iteraciones cerradas con una meta demostrable, no un flujo continuo.

Épicas por módulo

Base de datos, API, experiencia ciudadana, moderación y entrega.

Un dueño por ticket

Dependencias explícitas entre módulos con «is blocked by».

Definition of done

Mergeado en main, CI en verde y validado en la aplicación.

El tablero en cifras

61 actividades

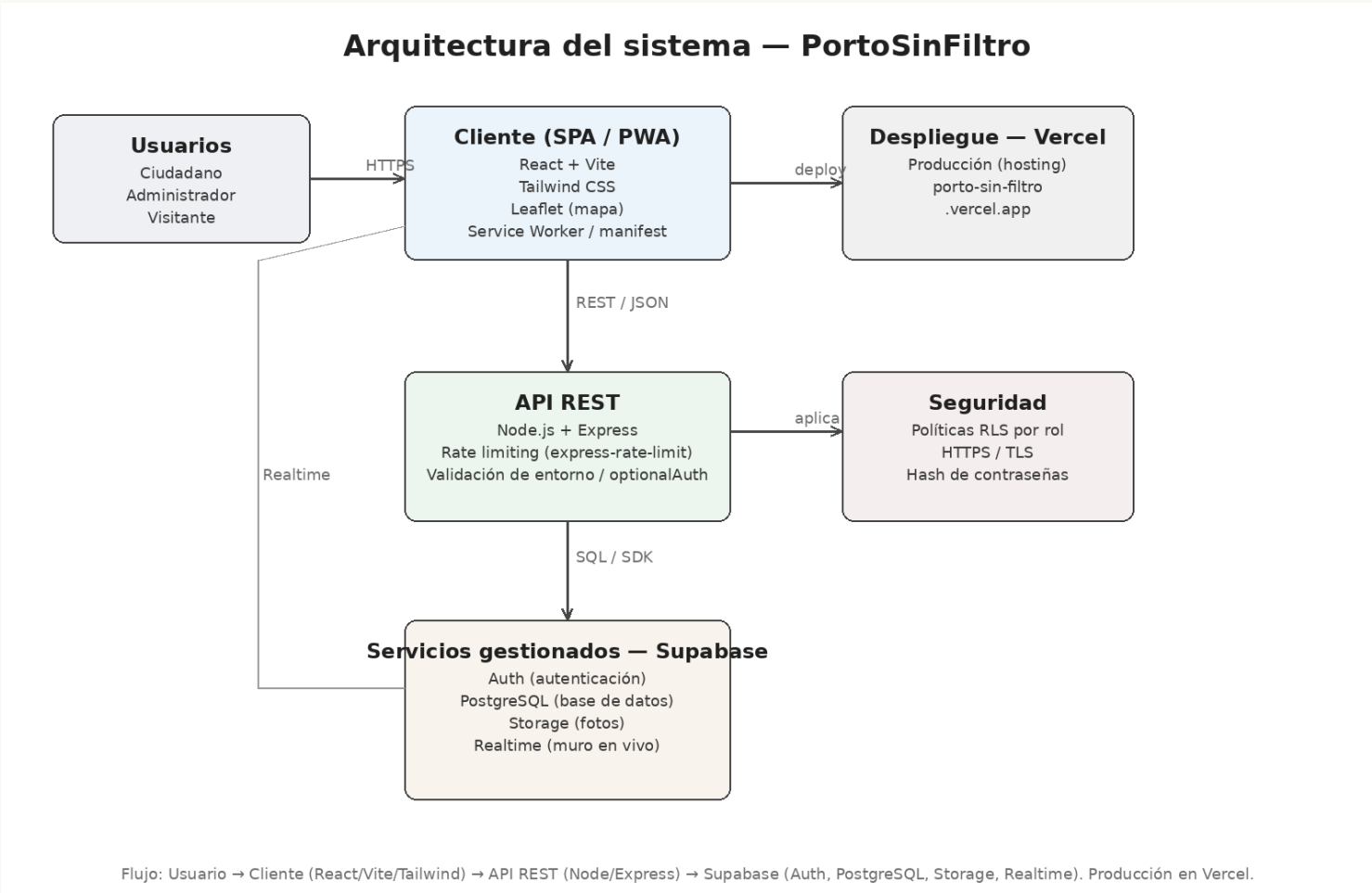
8 épicas

5 integrantes

Flujo de estados

-  Por hacer
-  En progreso
-  En revisión (PR + CI)
-  Finalizado

Tres capas, una regla



CLIENTE

React 18 + Vite + Tailwind. Muro, detalle, nueva denuncia, panel público, admin y PWA.

SERVIDOR

Node.js + Express. requireAuth() valida JWT ES256 vía JWKS, requireRol() autoriza, helmet, rate limit y express-validator.

DATOS

PostgreSQL en Supabase: tablas, vista_denuncias, RLS y trigger. Storage con bucket denuncias (JPG/PNG/WEBP).

Regla: el cliente nunca escribe directo en la base.

9 tablas, 2 vistas, 0 estados escritos a mano

Identidad

perfiles extiende auth.users; rol ciudadano o administrador, creado por trigger al registrarse.

Núcleo

denuncias con categoría, zona, gravedad, lat/lng, anonimato y bandera de moderación.

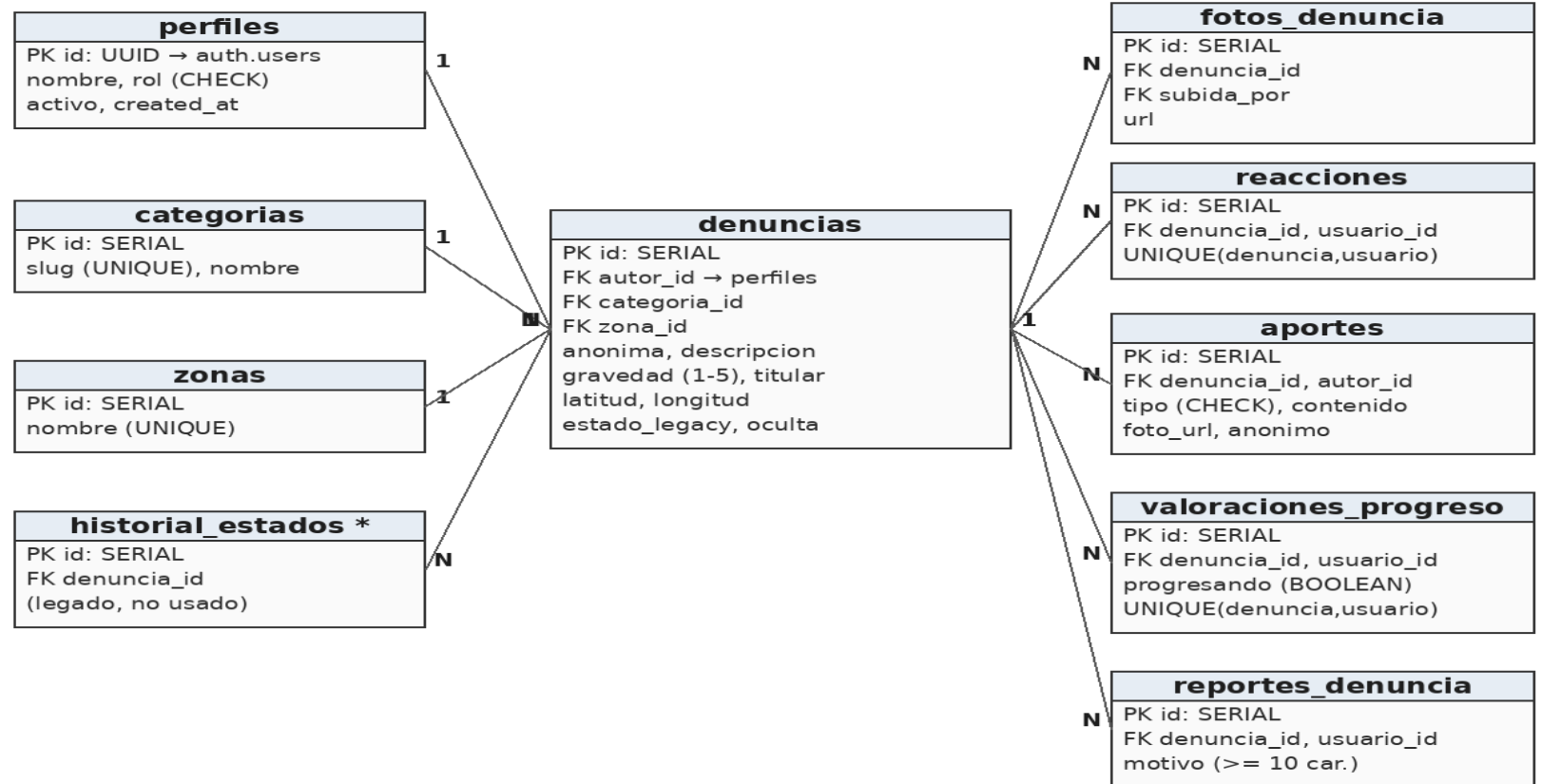
Evidencia

fotos_denuncia, aportes, reacciones, valoraciones_progreso y reportes_denuncia.

Lectura

vista_denuncias para el público y vista_denuncias_admin para moderación.

Modelo Entidad-Relación (DER)



* Tabla heredada: se conserva en el esquema, sin uso en el modelo comunitario actual.

El estado no se guarda: se calcula

No existe una columna estado editable: la vista SQL la deriva de la evidencia en cada consulta, así que nadie puede falsear un «resuelto».

```
-- vista_denuncias · se evalúa en cada lectura del muro

CASE

  WHEN resoluciones >= 3      THEN 'resuelta'

  WHEN progreso_si  >= 2

    AND progreso_si > progreso_no THEN 'con_avance'

  ELSE 'activa'

END AS estado
```

Ventaja

Cero desincronización: no hay estado que actualizar en ninguna tabla.

Costo asumido

Se recalcula al leer; con más volumen, pasaría a vista materializada.

Efecto

Ni el administrador puede marcar algo como resuelto: solo la comunidad.

ACTIVA

›

CON AVANCE

›

RESUELTA

3 aportes de resolución · 2 votos de progreso que superen a los negativos · todo verificado por usuario

Defensa en capas

1 • IDENTIDAD

JWT ES256

Login por email y contraseña en Supabase Auth. El backend verifica cada token contra el JWKS público con la librería jose: no hay secreto compartido que filtrar.

2 • AUTORIZACIÓN

RBAC por ruta

`requireRol('ciudadano')` y `requireRol('administrador')` protegen cada endpoint. Crear denuncias es exclusivo de ciudadanos; ocultar, de administradores.

3 • ENTRADA

Validación

`express-validator` en cada ruta, `helmet`, CORS por lista blanca de orígenes y `rate limit` más estricto en las escrituras.

4 • BASE DE DATOS

RLS

Políticas por fila en PostgreSQL. Verificadas con pruebas negativas: sin sesión no se inserta ni se leen perfiles ajenos.

Código modular, probado en cada push

48

casos en 13 archivos

28 de backend con Supertest · 20 de frontend con Testing Library

CI

GitHub Actions

Ejecuta las dos suites y el build del frontend en cada push y PR a main

4

módulos de rutas

Separados por dominio, más middleware/ y db/ aislados

3

capas en frontend

pages/ · components/ · lib/ con API, constantes y realtime

Qué cubren las pruebas

- ✓ Permisos por rol: respuestas 401 y 403
- ✓ CRUD completo de denuncias
- ✓ Moderación con paginación
- ✓ Activar y desactivar usuarios
- ✓ Cálculo del estado comunitario
- ✓ Límite de peticiones: 429 al superar el umbral

Cómo se ejecutan

- | | |
|--------------------------------------|---|
| <code>npm test</code> | en backend/ · 7 archivos, 28 pruebas |
| <code>npm test</code> | en frontend/ · 6 archivos, 20 pruebas |
| <code>npm run test:rate-limit</code> | prueba de carga contra la API |
| <code>npm run verificar:db</code> | verifica el esquema y las políticas RLS |

En vivo, no en localhost

porto-sin-filtro.vercel.app

github.com/castro-bot/portoSinFiltro

Dominio público con HTTPS · frontend y API en el mismo despliegue vía vercel.json

Vercel

Build del frontend y funciones serverless para /api/*. Despliegue automático desde main.

Supabase Cloud

PostgreSQL gestionado, Auth, Storage y Realtime, con políticas RLS activas.

Docker

backend/Dockerfile y docker-compose.yml para levantarlo en un servidor propio.

Secretos

Variables de entorno en la plataforma. La service key nunca se versiona ni llega al navegador.

CORS

FRONTEND_URL admite varios orígenes separados por coma: producción y desarrollo.

Realtime

El muro se actualiza en vivo; requiere aplicar la migración de Realtime en Supabase.

PWA

Instalable, con service worker verificado con la red completamente apagada.

Lo entregado y lo siguiente

ENTREGADO

- ✓ Muro público, detalle y creación con mapa y foto
- ✓ Registro y login de ciudadanos con roles
- ✓ Estado comunitario calculado y actualizado en vivo
- ✓ Moderación de administrador y gestión de usuarios
- ✓ Panel público de estadísticas y mapa agregado
- ✓ PWA instalable, pruebas automatizadas y CI

SIGUIENTE ITERACIÓN

1

Notificaciones

Avisar al autor cuando su caso cambia de estado.

2

Reporte de cobertura

Y pruebas de integración de las páginas.

3

Capa de servicios

Separar el SQL del HTTP en el backend.

4

Vista materializada

Del estado, si crece el volumen de denuncias.

Gracias